

UNIVERSITATEA TEHNICĂ A MOLDOVEI

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

LUCRARE DE LABORATOR NR. 2

Meet the OS You Will Build

Disciplina: Sisteme de Operare

A efectuat:
student gr. FAF-243
Daniel Nicorici

Au verificat:
asist. univ. Patricia Reițman
asist. univ. Marius Dumitrașcu

Chișinău — 2026

Cuprins

Purpose of the work **2**

1 Part 1. Build it and boot it **2**

2 Part 2. Use xv6 as the Unix it is **2**

3 Part 3. Read the source **3**

3.1 System calls in user/cat.c 3

3.2 Where the kernel implements read and write 4

3.3 How a call crosses from user/ into kernel/ 4

4 Part 4. Write your first xv6 program **5**

4.1 Implementation 5

4.2 Adding the program to the build 6

4.3 Testing 6

Conclusions **6**

Purpose of the work

The purpose of this lab is to build xv6, a small but real Unix written in about 9 000 lines of C, from source; to boot it on an emulated RISC-V machine in QEMU; to use it as the tiny Unix it is; to read both sides of a system call (the program in `user/` that asks and the kernel code in `kernel/` that answers); and to add a first program of my own, `sleep`, that makes a real system call into the kernel I compiled.

Environment: xv6 source `mit-pdos/xv6-riscv` at commit `06aad25` (2026-09-26), cross-compiled with `riscv64-linux-gnu-gcc` 16.2.1 and run on QEMU 11.1.1 (`qemu-system-riscv64`), on my own laptop running Arch Linux.

1 Part 1. Build it and boot it

```
$ cd ~/xv6-riscv
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -
    ↪ global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device
    ↪ virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

`make qemu` compiles the kernel into `kernel/kernel`, compiles every user program listed in `UPROGS`, packs `README` and those programs into the disk image `fs.img`, and starts QEMU with that kernel and disk. The emulated machine is the RISC-V `virt` board with 128 MB of RAM and 3 harts (hardware threads, i.e. CPU cores): hart 0 boots the kernel, harts 1 and 2 report that they are starting, and then the first process, `init`, starts the shell. QEMU is left with `Ctrl-A` followed by `X`.

2 Part 2. Use xv6 as the Unix it is

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 35912
echo           2 4 34808
forktest       2 5 17704
grep           2 6 43336
init           2 7 35152
kill           2 8 34704
ln             2 9 34504
ls             2 10 41992
mkdir          2 11 34760
rm             2 12 34736
sh             2 13 57120
stressfs       2 14 35584
usertests      2 15 201568
```

```
grind          2 16 50384
wc             2 17 36880
zombie         2 18 34056
logstress      2 19 36728
forphan        2 20 35520
dorphan        2 21 34976
sync           2 22 34152
sleep          2 23 34480
console        3 24 0
$ echo hello xv6
hello xv6
$ ls | grep c
cat            2 3 35912
echo           2 4 34808
wc             2 17 36880
sync           2 22 34152
console        3 24 0
$ wc README
48 336 2441 README
```

The columns printed by xv6's `ls` are: name, type (1 = directory, 2 = file, 3 = device), inode number and size in bytes. `console` has type 3: the terminal is a device file, just like `/dev/tty` on Linux.

Observations:

1. **Three programs xv6 ships with.** `ls`, `cat` and `grep`; the image also contains `echo`, `wc`, `sh`, `mkdir`, `rm`, `kill`, `ln` and the test programs `usertests` and `forktest`.
2. **What a pipe needs.** First, *processes*: the shell uses `fork` and `exec` to create two processes, one running `ls` and one running `grep`, which run at the same time. Second, *inter-process communication*: the `pipe` system call creates a kernel buffer with a write end and a read end, and the shell uses `close` and `dup` to connect the standard output of `ls` to the write end and the standard input of `grep` to the read end (`user/sh.c`, lines 103–113).
3. **xv6 shell vs. Linux shell.** For simple commands the xv6 shell behaves like bash (programs, pipes, >/< redirection, `;` and `&`), but it is much more minimal: no history, no Tab completion, no variables, no `*` globbing, and the prompt is just `$`.

3 Part 3. Read the source

3.1 System calls in user/cat.c

```c
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"

char buf[512];

void
cat(int fd)
{
  int n;

  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
```

```
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if (n < 0) {
    fprintf(2, "cat: read error\n");
    exit(1);
  }
}
```

3.2 Where the kernel implements read and write

```
$ grep -n "sys_read\|sys_write" kernel/sysfile.c
69:sys_read(void)
83:sys_write(void)
```

```
uint64
sys_read(void)
{
  struct file *f;
  int n;
  uint64 p;

  argaddr(1, &p);
  argint(2, &n);
  if (argfd(0, 0, &f) < 0)
    return -1;
  return fileread(f, p, n);
}
```

3.3 How a call crosses from user/ into kernel/

On the user side, `read` is not C code at all. The script `user/usys.pl` generates a three-instruction stub for every system call in `user/usys.S`: it loads the call number into register `a7` and executes `ecall`, which traps into the kernel.

```
$ grep -A3 "^read:" user/usys.S
read:
 li a7, SYS_read
 ecall
 ret
```

In the kernel, `syscall()` reads that number back from the saved registers and calls the matching function from its table (`[SYS_read] = sys_read` on line 115, with `SYS_read` = 5 defined in `kernel/syscall.h`). The return value is stored in `a0`, which is where the user program finds it.

```
void
syscall(void)
{
  int num;
  struct proc *p = myproc();

```

```
142   num = p->trapframe->a7;
143   if (num > 0 && num < NELEM(syscalls) && syscalls[num]) {
144     // Use num to lookup the system call function for num, call it,
145     // and store its return value in p->trapframe->a0
146     p->trapframe->a0 = syscalls[num]();
147   } else {
148     printk("%d %s: unknown sys call %d\n", p->pid, p->name, num);
149     p->trapframe->a0 = -1;
150   }
151 }
```

Observations:

1. **System calls used by `user/cat.c`.** `open(path, O_RDONLY)` asks the kernel to find a file by name and return a file descriptor; `read(fd, buf, 512)` asks it to copy up to 512 bytes from that file into `buf` (it returns the number of bytes, 0 at end of file, -1 on error); `write(1, buf, n)` asks it to copy `n` bytes to file descriptor 1, the console; `close(fd)` releases the descriptor; `exit(status)` ends the process and passes the status to the parent's `wait`. `fprintf` is not a system call: it is a library function in `user/printf.c` that formats the text and then calls `write` on descriptor 2.
2. **Where `sys_read` is implemented.** In `kernel/sysfile.c`, line 69 (with its return type `uint64` on line 68); `sys_write` follows on line 83.
3. **`kernel/` vs. `user/`.** The code in `kernel/` runs in privileged (supervisor) mode with access to all memory and hardware and provides the services; the code in `user/` runs as ordinary, isolated programs that can only obtain those services by asking the kernel through system calls.

4 Part 4. Write your first xv6 program

4.1 Implementation

In August 2025 the xv6 authors renamed the user system call `sleep()` to `pause()` (upstream commit `f5dea58`, which also renamed kernel functions such as `fork` to `kfork` and `exit` to `kexit`). On the current tree `user/user.h` declares only `int pause(int);`, so the handout's version, which calls `sleep()`, does not build. My program therefore calls `pause()`:

```
1  #include "kernel/types.h"
2  #include "user/user.h"
3
4  int
5  main(int argc, char *argv[])
6  {
7    if(argc != 2){
8      fprintf(2, "usage: sleep <ticks>\n");
9      exit(1);
10   }
11   pause(atoi(argv[1]));
12   exit(0);
13 }
```

`argv[0]` is the program name and `argv[1]` the number of ticks, so any other argument count is a usage error: the message goes to file descriptor 2 (standard error) and `exit(1)` reports the failure

to the parent shell.

4.2 Adding the program to the build

```
$ git diff Makefile
diff --git a/Makefile b/Makefile
index af1e7a5..089e215 100644
--- a/Makefile
+++ b/Makefile
@@ -150,6 +150,7 @@ UPROGS=\
        $U/_forphan\
        $U/_dorphan\
        $U/_sync\
+       $U/_sleep\

 fs.img: mkfs/mkfs README $(UPROGS)
        mkfs/mkfs fs.img README $(UPROGS)
```

User programs are built as `user/_name`; `mkfs` strips the leading underscore when it copies them into `fs.img` (`mkfs/mkfs.c`, line 147), so inside xv6 the program is simply `sleep`. Without the `UPROGS` line the program is never compiled or copied, and the shell answers `exec sleep failed`.

4.3 Testing

```
$ sleep 10
$ sleep
usage: sleep <ticks>
$ sleep abc 2
usage: sleep <ticks>
```

To check that the program really waits rather than returning at once, I timed it from the host: after typing `sleep 50; echo done` the prompt line appeared at 41.46 s and `done` at 46.42 s, a pause of 4.96 s. One xv6 tick is about 0.1 s (the kernel arms the next timer interrupt 1 000 000 cycles ahead, `kernel/trap.c`, line 179), so 50 ticks $\approx$ 5 s, as expected.

Conclusions

I built xv6 from source, booted it in QEMU and used it as a small Unix with the same basic tools and pipes as Linux. Reading the source showed the split at the heart of every operating system: programs in `user/` only ask, and the kernel in `kernel/` does the privileged work.

My `sleep` program crosses that boundary for real. When it calls `pause(10)`, the argument goes into register `a0`, and the stub in `user/usys.S` loads the call number 13 into `a7` and executes `ecall`. The CPU switches to supervisor mode, `uservec` saves the user registers, `usertrap()` recognises a system call, and `syscall()` uses `a7` to call `sys_pause`, which waits for ten timer ticks. The result travels back in `a0`, the CPU returns to user mode, and the program exits. Every service a program gets from the OS, from reading a file to sleeping, follows this same path.